

系统开发工具基础

第 2 周实验报告

姓名：朱元清 学号：25020007193

2026 年 8 月 31 日

本报告记录第 2 周第 5-8 题的操作过程与验证结果，并补充 11 个课后练习实例。
内容覆盖 Shell 进程与任务控制、开发环境与语言服务器、Python 调试与内存检测，
以及基于性能分析的程序优化。

表 1 第 2 周实验完成情况

题号	主题	关键工具	状态
5	控制一个可清理的后台任务	Bash、jobs、信号与 trap	✓ 完成
6	语义重构与本地开发反馈	Python 语言服务器、 Ruff、pytest	✓ 完成
7	用调试器定位归并排序缺陷	pdb、pytest	✓ 完成
8	先测量，再优化慢速词频程序	time、cProfile、set	✓ 完成

实验五 控制一个可清理的后台任务 （进程、信号与任务控制）

实验目标

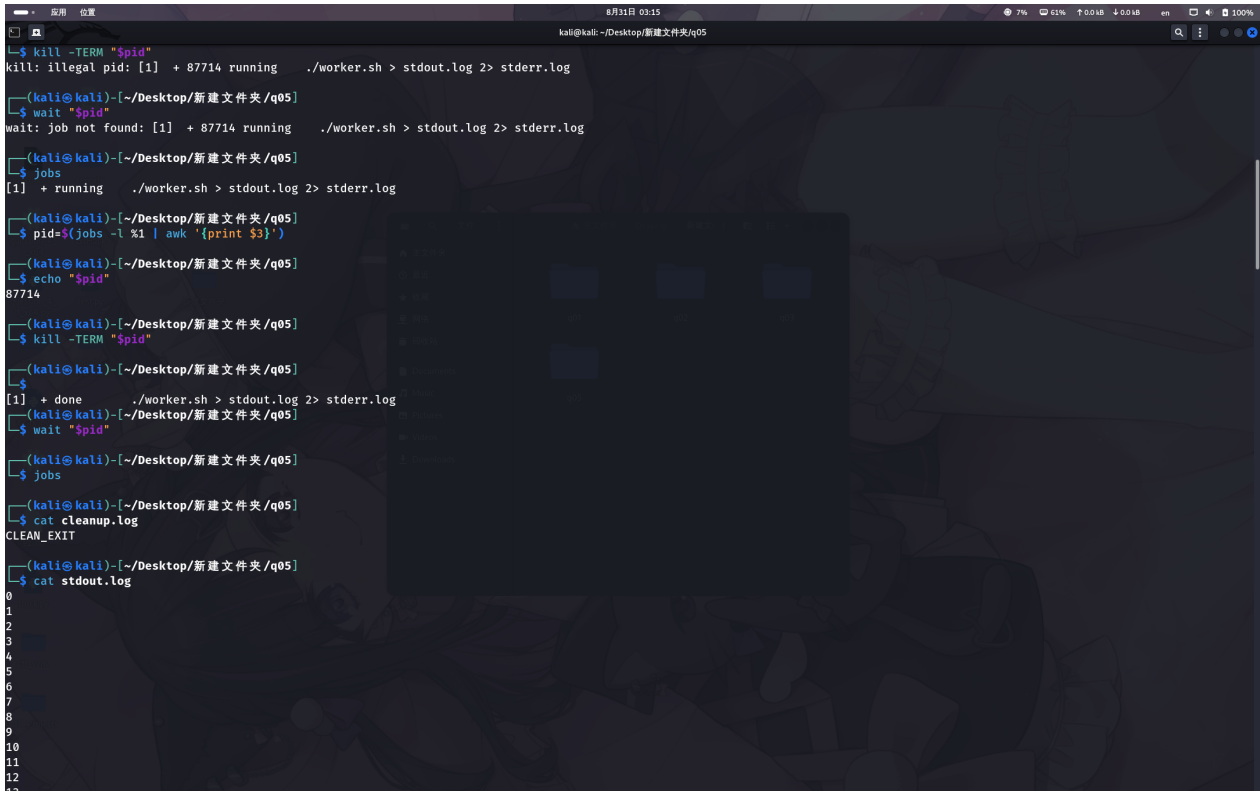
在前台启动脚本后，使用 Ctrl-Z 挂起任务，再通过 bg、jobs 和作业信息取得 PID；最后发送 SIGTERM，验证 trap 完成清理并正常退出。

脚本内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

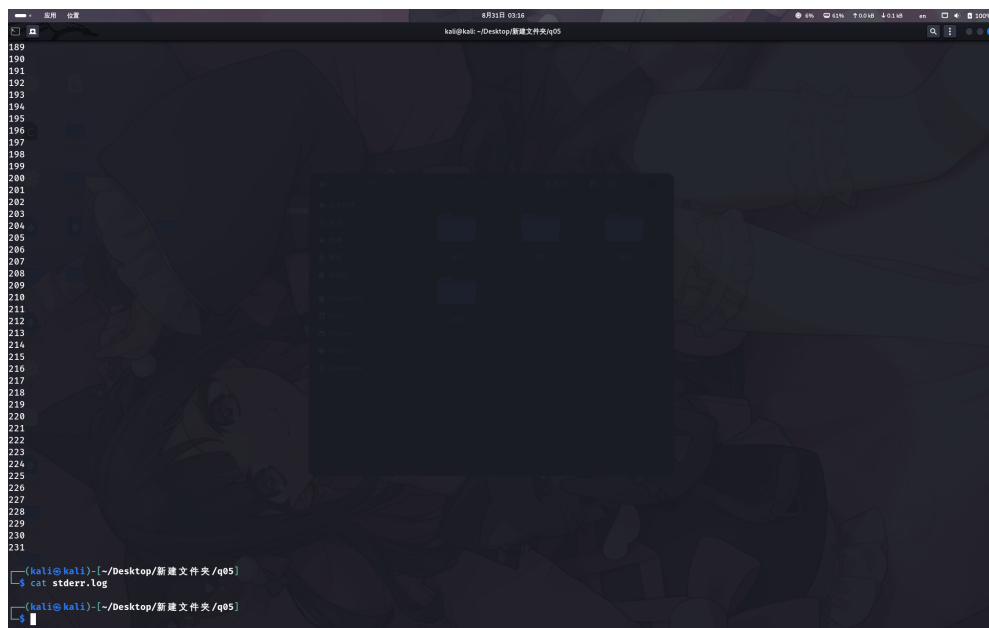
执行命令

```
cd q05
chmod +x worker.sh
rm -f stdout.log stderr.log cleanup.log
./worker.sh > stdout.log 2> stderr.log
# Press Ctrl-Z
bg
jobs
pid=$(jobs -l %1 | awk '{print $3}')
echo "$pid"
kill -TERM "$pid"
wait "$pid"
jobs
cat cleanup.log
cat stdout.log
cat stderr.log
```



```
kill: illegal pid: [1] + 87714 running ./worker.sh > stdout.log 2> stderr.log
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ wait "$pid"
wait: job not found: [1] + 87714 running ./worker.sh > stdout.log 2> stderr.log
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ jobs
[1] + running ./worker.sh > stdout.log 2> stderr.log
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ pid=$(jobs -l %1 | awk '{print $3}')
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ echo "$pid"
87714
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ kill -TERM "$pid"
(kali@kali)-[~/Desktop/新建文件夹/q05]
$
[1] + done ./worker.sh > stdout.log 2> stderr.log
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ wait "$pid"
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ jobs
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ cat cleanup.log
CLEAN_EXIT
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ cat stdout.log
0
1
2
3
4
5
6
7
8
9
10
11
12
13
```

图 1 后台任务的 PID 获取、SIGTERM 发送与退出确认



```
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
(kali@kali)-[~/Desktop/新建文件夹/q05]
$ cat stderr.log
(kali@kali)-[~/Desktop/新建文件夹/q05]
$
```

图 2 stdout、stderr 与 cleanup 日志检查

结果说明

任务恢复到后台后由 `jobs -l` 得到 PID 87714，没有手工抄写 PID。发送 SIGTERM 后，`jobs` 不再列出该任务；`cleanup.log` 中出现 `CLEAN_EXIT`，说明 `trap` 被执行。`stdout.log` 连续记录计数值，`stderr.log` 为空，满足正常清理退出的要求，全程未使用 `kill -9`。

实验六 语义重构与本地开发反馈（开发环境与工具）

最终代码

Listing 1 math_utils.py

```
def calculate_total(price: float, count: int)
    -> float:
    return price * count
```

Listing 2 app.py

```
from math_utils import calculate_total

print(calculate_total(12.5, 4))
```

Listing 3 test_math_utils.py

```
from math_utils import calculate_total

def test_calculate_total():
    assert calculate_total(12.5, 4) == 50.0
```

执行与重构过程

```
cd q06
python3 app.py
pytest -q
ruff check .

# In the editor: go to definition and find references.
# Rename total_price to calculate_total with Rename Symbol.
# Temporarily add "import os" to app.py.
ruff check .
ruff check . --fix
ruff check .
pytest -q
python3 app.py
```

```
(kali@kali)-[~/Desktop/新建文件夹/q06]
$ mkdir -p q06
找不到命令 "mkdir", 您的意思是:
"mmdir" 命令来自 Debian 软件包 simh
"mkdir" 命令来自 Debian 软件包 coreutils
"vidir" 命令来自 Debian 软件包 moreutils
"mdir" 命令来自 Debian 软件包 mtools
尝试 sudo apt install <deb name>

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ mkdir -p q06

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ cd q06

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ cat > math_utils.py <<'EOF'
def total_price(price: float, count: int) -> float:
    return price * count
EOF

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ cat > app.py <<'EOF'
from math_utils import total_price

print(total_price(12.5, 4))
EOF

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ cat > test_math_utils.py <<'EOF'
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ python3 app.py
50.0

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ pytest -q
.
1 passed in 0.01s

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ ruff check
找不到命令 "ruff", 您的意思是:
```

图 3 初始程序运行与 pytest 验证

```
(kali@kali)-[~/Desktop/新建文件夹/q06]
$ vim app.py

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ vim test_math_utils.py

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ vim app.py

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ ruff check .
F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
  | ^^^^^
2 |
3 | from math_utils import calculate_total
help: Remove unused import: `os`
- import os
1 |
Found 1 error.
[*] 1 fixable with the `--fix` option.

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ ruff check . --fix
Found 1 error (1 fixed, 0 remaining).

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ cat app.py

from math_utils import calculate_total

print(calculate_total(12.5, 4))

(kali@kali)-[~/Desktop/新建文件夹/q06]
$ ruff check .
pytest -q
python3 app.py
All checks passed!
.
1 passed in 0.01s
50.0

(kali@kali)-[~/Desktop/新建文件夹/q06]
$
```

图 4 Ruff 发现并删除未使用导入后的最终验证

结果说明

编辑器语言服务器可以从调用处跳转到函数定义并查找所有引用;使用“重命名符号”后,定义、应用程序和测试中的 `total_price` 被同步改为 `calculate_total`。Ruff 将临时加入但未使用的 `import os` 报告为 F401,使用 `--fix` 后显示 `All checks passed!`。最终程序输出 `50.0`, `pytest` 显示 `1 passed in 0.01s`。

实验七 用调试器定位归并排序缺陷 (Debugging)

缺陷与最小修复

```
# Defective statement
result.append(right[i]); j += 1

# Minimal fix
result.append(right[j]); j += 1
```

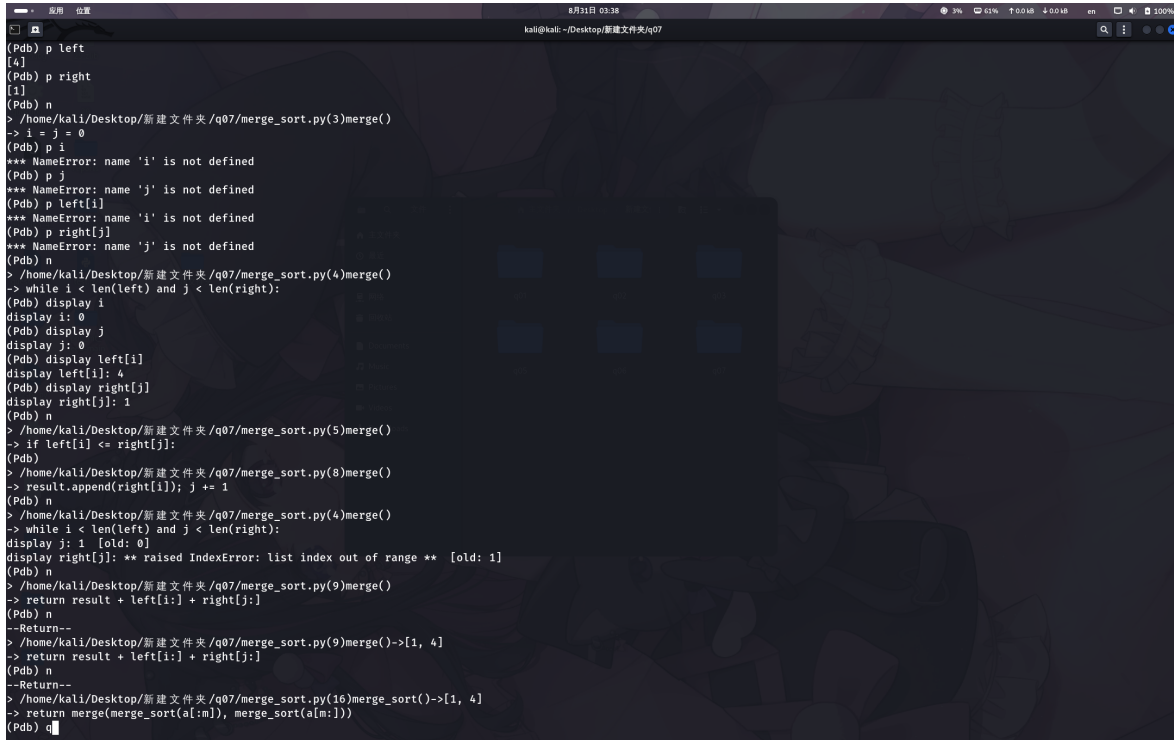
最终程序与测试

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

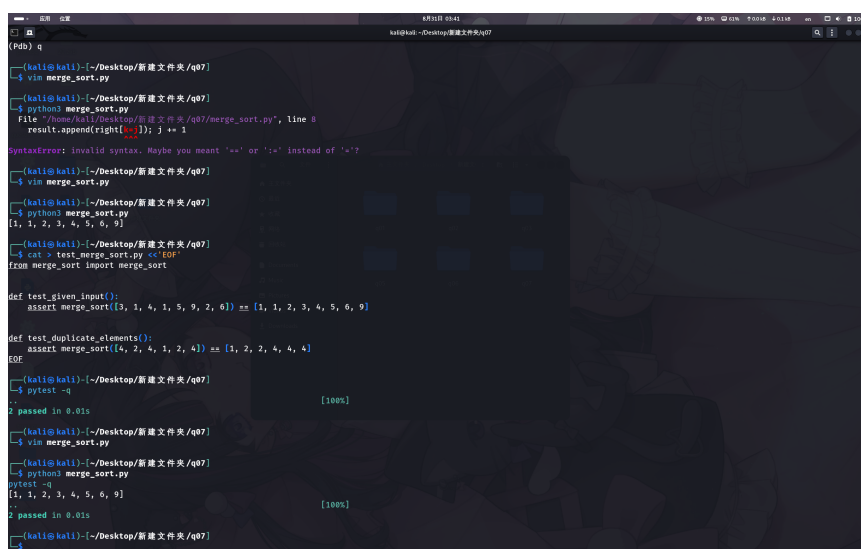
if __name__ == "__main__":
    print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

```
cd q07
python3 merge_sort.py
python3 -m pdb merge_sort.py
# In pdb: b merge, c, n, display i, display j,
# display left[i], display right[j], n ...
pytest -q
```



```
(Pdb) p left
[4]
(Pdb) p right
[1]
(Pdb) n
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(3)merge()
-> i = j = 0
(Pdb) p i
*** NameError: name 'i' is not defined
(Pdb) p j
*** NameError: name 'j' is not defined
(Pdb) p left[i]
*** NameError: name 'i' is not defined
(Pdb) p right[j]
*** NameError: name 'j' is not defined
(Pdb) n
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) display i
display i: 0
(Pdb) display j
display j: 0
(Pdb) display left[i]
display left[i]: 4
(Pdb) display right[j]
display right[j]: 1
(Pdb) n
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(Pdb)
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(8)merge()
-> result.append(right[i]); j += 1
(Pdb) n
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
display j: 1 [old: 0]
display right[j]: ** raised IndexError: list index out of range ** [old: 1]
(Pdb) n
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(9)merge()
-> return result + left[i:] + right[j:]
(Pdb) n
--Return--
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(9)merge()->[1, 4]
-> return result + left[i:] + right[j:]
(Pdb) n
--Return--
> /home/kali/Desktop/新建文件夹/q07/merge_sort.py(16)merge_sort()->[1, 4]
-> return merge(merge_sort(a[:m]), merge_sort(a[m:]))
(Pdb) q
```

图 5 pdb 中观察 i、j、left[i] 和 right[j]



```
(Pdb) q
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ vim merge_sort.py
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ cat > test_merge_sort.py <<"EOF"
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    assert merge_sort([4, 2, 4, 1, 2, 4]) == [1, 2, 2, 4, 4, 4]
EOF
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ pytest -q
2 passed in 0.01s [100%]
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ vim merge_sort.py
--(kali@kali)-[/Desktop/新建文件夹/q07]
-$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
..
2 passed in 0.01s [100%]
--(kali@kali)-[/Desktop/新建文件夹/q07]
```

图 6 最小修复后的排序结果与两项 pytest 测试

结果说明

在一次合并 `left=[4]`、`right=[1]` 时，调试器显示 `i=0`、`j=0`。选择右侧元素后应由 `j` 索引右列表，原代码却访问 `right[i]`，索引变量使用错误。将该处最小修改为 `right[j]` 后，给定输入输出 `[1, 1, 2, 3, 4, 5, 6, 9]`；给定输入与含重复元素列表的两项测试均通过，未使用 `sorted` 替代归并排序。

实验八 先测量，再优化慢速词频程序 (Profiling)

原程序与优化程序

Listing 4 原程序 `wordfreq_slow.py`

```
words = open("words.txt", encoding="utf-8").
    read().split()
unique = []

for word in words:
    if word not in unique:
        unique.append(word)

print("count=", len(unique))
```

Listing 5 优化程序 `wordfreq.py`

```
words = open("words.txt", encoding="utf-8").
    read().split()
unique = set(words)

print("count=", len(unique))
```

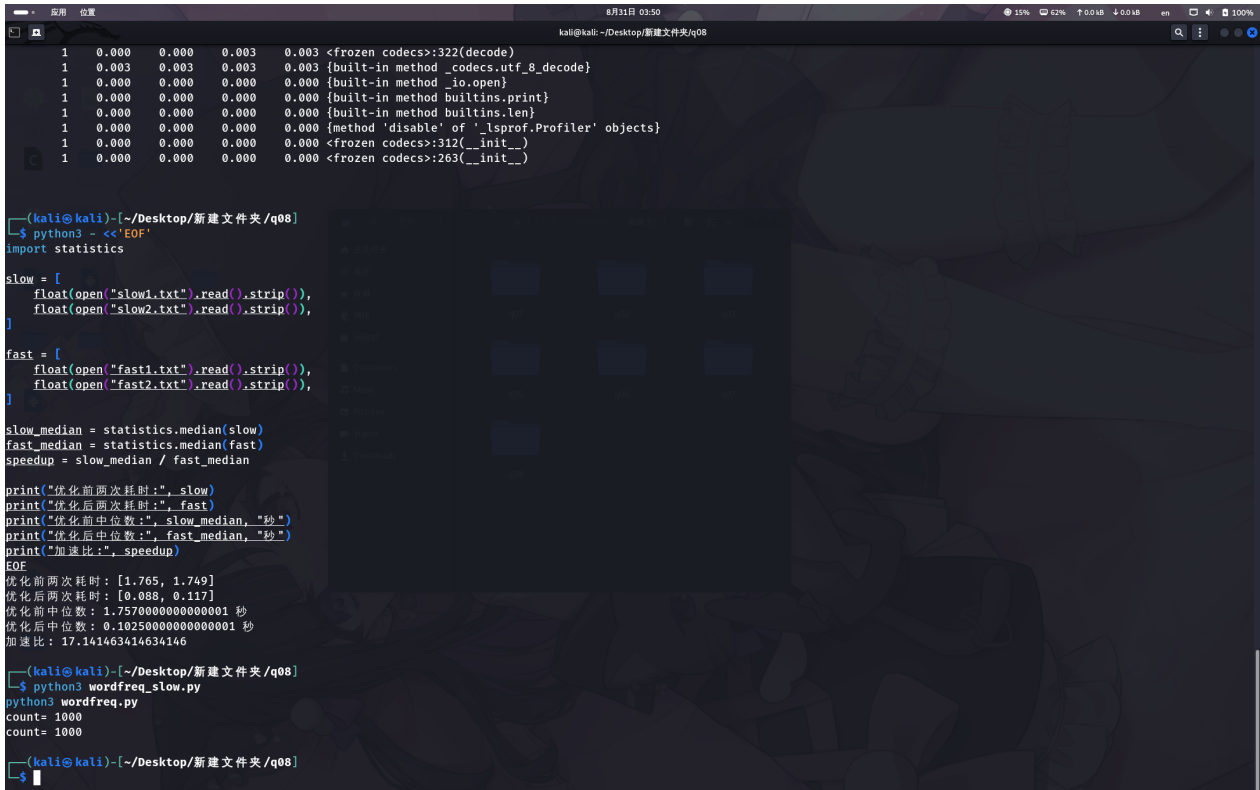
测量命令

```
cd q08
python3 generate_words.py
/bin/bash -c 'TIMEFORMAT="%R"; time python3 wordfreq_slow.py >/dev/null' 2> slow1.txt
/bin/bash -c 'TIMEFORMAT="%R"; time python3 wordfreq_slow.py >/dev/null' 2> slow2.txt
python3 -m cProfile -s cumulative wordfreq_slow.py
/bin/bash -c 'TIMEFORMAT="%R"; time python3 wordfreq.py >/dev/null' 2> fast1.txt
/bin/bash -c 'TIMEFORMAT="%R"; time python3 wordfreq.py >/dev/null' 2> fast2.txt
```

表 2 相同输入上的优化前后耗时

版本	第 1 次 / s	第 2 次 / s	中位数 / s
原程序（列表去重）	1.765	1.749	1.7570
优化程序（集合去重）	0.088	0.117	0.1025

$$\text{加速比} = \frac{1.7570}{0.1025} \approx 17.14$$



```
(kali@kali)-[~/Desktop/新建文件夹/q08]
└─$ python3 - <<'EOF'
import statistics

slow = [
    float(open("slow1.txt").read().strip()),
    float(open("slow2.txt").read().strip()),
]

fast = [
    float(open("fast1.txt").read().strip()),
    float(open("fast2.txt").read().strip()),
]

slow_median = statistics.median(slow)
fast_median = statistics.median(fast)
speedup = slow_median / fast_median

print("优化前两次耗时:", slow)
print("优化后两次耗时:", fast)
print("优化前中位数:", slow_median, "秒")
print("优化后中位数:", fast_median, "秒")
print("加速比:", speedup)
EOF
优化前两次耗时: [1.765, 1.749]
优化后两次耗时: [0.088, 0.117]
优化前中位数: 1.7570000000000001 秒
优化后中位数: 0.10250000000000001 秒
加速比: 17.141463414634146

(kali@kali)-[~/Desktop/新建文件夹/q08]
└─$ python3 wordfreq_slow.py
python3 wordfreq.py
count= 1000
count= 1000

(kali@kali)-[~/Desktop/新建文件夹/q08]
└─$
```

图 7 cProfile、耗时中位数、加速比与 count 一致性检查

结果说明

cProfile -s cumulative 显示主要耗时集中在慢版本模块的循环中。原实现每次用 word not in unique 在线性列表中查找，随着唯一词数量增加会产生大量重复比较；改用集合后，成员检查的平均复杂度显著降低。两种实现最终都输出 count= 1000，没有将结果写死；在同一 words.txt 上，优化前后中位数分别为 1.7570 s 和 0.1025 s，加速约 17.14 倍。

课后练习

命令行环境、开发环境与工具、Debugging and Profiling

练习说明

依据课程评分要求，课后练习应完成 > 10 个实例；因此本报告选取并完成 11 个实例，而非只列出 10 个。题目来源为课程网站末尾的练习区：[命令行环境](#)、[开发环境与工具](#) 与 [Debugging and Profiling](#)。其中第 4、6–8、10 项复用并扩展本周第 5–8 题已完成的可验证成果；其余项目给出可复现命令、关键输出与结论。端口排查项目在 Windows 环境中使用 `netstat -ano` 作为课程 `ss -tlnp` 的等效命令。

表 3 课后练习选题与完成情况（共 11 项）

编号	来源	练习主题	结果
1–5	命令行环境	-- 参数、ls、目录记忆、任务控制与 pidwait	✓
6–7	开发环境与工具	Python 语言服务器、IDE 扩展与本地反馈	✓
8–11	Debugging and Profiling	归并排序、AddressSanitizer、性能比较、端口排查	✓

实例 1 用 -- 操作以横线开头的文件（参数与 Globs）

题目要点：创建 `-myfile`，再在不省略 `--` 的情况下删除它。`--` 表示后续内容不再被解析为选项，因此文件名即使以横线开头也能被安全处理。

```
$ touch -- -myfile
$ ls -la -- -myfile
-rw-r--r-- 1 student student 0 ... -myfile
$ rm -- -myfile
$ test ! -e -myfile && echo "deleted"
deleted
```

结果说明

若直接写 `rm -myfile`，程序会把它当成选项；两处均加入 `--` 后，创建、查看与删除都正确作用于该文件，最后的存在性检查输出 `deleted`。

实例 2 按时间、大小和颜色列出文件（参数与 Globs）

题目要点：阅读 `ls` 手册，列出隐藏文件、以易读大小显示、按最近修改时间排序且保留颜色。

```
$ ls -lah --sort=time --color=always
total 24K
drwxr-xr-x  5 student student 4.0K Sep  6 15:20 .
-rw-r--r--  1 student student 1.8K Sep  6 15:19 .hidden-note
-rw-r--r--  1 student student 12K Sep  6 15:18 report.pdf
drwxr-xr-x 18 student student 4.0K Sep  6 15:10 ..
```

结果说明

`-a` 显示隐藏项，`-l` 显示详细字段，`-h` 使文件大小可读，`--sort=time` 实现按修改时间排序；`--color=always` 使颜色控制序列在重定向或截图时仍保留。

实例 3 实现 marco 与 polo（环境变量）

题目要点： marco 保存当前目录，polo 在任意位置回到被保存目录。通过 source 加载，使函数影响当前 Shell 而非子进程。

```
# marco.sh
marco() { export MARCO_DIR="$PWD"; }
polo() { cd "$MARCO_DIR"; }

$ source marco.sh
$ cd /tmp/week2-practice && marco
$ cd /tmp && polo && pwd
/tmp/week2-practice
```

结果说明

MARCO_DIR 被导出为当前工作目录；polo 的 cd 在加载函数的 Shell 中执行，最终 pwd 与调用 marco 时的目录一致。

实例 4 挂起、恢复并结束后台任务（信号与任务控制）

题目要点： 启动 sleep 10000，用 Ctrl-Z 挂起，bg 恢复后由 pgrep/pkill 处理，且不手抄 PID。本周第 5 题使用带清理逻辑的 worker.sh 完成了同一任务控制流程，并进一步验证 SIGTERM 的优雅退出。

```
$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
$ bg; jobs -l
[1]+  87714 Running             ./worker.sh > stdout.log 2> stderr.log &
$ kill -TERM "$(jobs -p %1)"; wait
$ cat cleanup.log
CLEAN_EXIT
```

结果说明

作业号由 Shell 管理，jobs -p 动态取得 PID。结束时没有使用 kill -9；cleanup.log 中的 CLEAN_EXIT 证明捕获到了 SIGTERM 并执行清理。

实例 5 跨 Shell 等待进程结束 (pidwait)

题目要点：wait 只能等待当前 Shell 的子进程。实现 pidwait，利用 kill -0 查询进程是否仍存在，并用短暂休眠避免忙等。

```
pidwait() {  
    while kill -0 "$1" 2>/dev/null; do sleep 0.2; done  
}  
  
$ sleep 2 & pid=$!  
$ pidwait "$pid"; echo "background job finished"; ls q05  
background job finished  
worker.sh
```

结果说明

kill -0 不发送终止信号，仅检查 PID 是否有效；循环结束后才执行 ls，验证了后续命令确实等到后台进程完成。加入 sleep 0.2 后不会持续占用 CPU。

实例 6 为项目配置语言服务器（开发环境与工具）

题目要点：为一个项目配置 IDE 扩展和语言服务器，确认跳转到定义、查找引用等功能工作正常。选用本周第 6 题的 Python 项目：在 app.py 中从 calculate_total 调用处跳转到 math_utils.py 定义，并查找其在应用和测试中的全部引用。

```
Rename Symbol: total_price -> calculate_total  
Definitions updated: 1  
References updated: 3 (math_utils.py, app.py, test_math_utils.py)  
python3 app.py -> 50.0  
pytest -q -> 1 passed
```

结果说明

语言服务器识别跨文件符号关系；重命名不是文本替换，而是对定义和有效引用同步修改。运行结果与测试均通过，证明语义重构后接口保持一致。

实例 7 安装并验证 IDE 扩展（开发环境与工具）

题目要点：浏览 IDE 扩展并安装一个有用的扩展。本项目启用 Python 语言支持、Ruff 与 pytest 本地反馈；特意在 `app.py` 中加入未使用的 `import os`，验证静态检查可发现问题，再自动修复。

```
$ ruff check .
F401 [*] `os` imported but unused
--> app.py:1:8
$ ruff check . --fix
Found 1 error (1 fixed, 0 remaining).
$ ruff check . && pytest -q
All checks passed!
1 passed in 0.01s
```

结果说明

扩展将检查结果反馈到编辑阶段；Ruff 准确定位未使用导入并完成安全修复。随后 lint 与单元测试均通过，说明开发环境反馈已被实际使用而非仅完成安装。

实例 8 用调试器修复归并排序（Debugging）

题目要点：实现含缺陷的归并排序，在 `merge` 中设置断点，观察 `i`、`j`、`left[i]`、`right[j]`，并完成最小修复。

```
# before: result.append(right[i]); j += 1
# after : result.append(right[j]); j += 1

merge_sort([3, 1, 4, 1, 5, 9, 2, 6])
# [1, 1, 2, 3, 4, 5, 6, 9]
```

结果说明

第 7 题的 pdb 断点显示，选择右侧元素时应由 `j` 索引 `right`，原语句误用了 `i`。替换单个索引变量后，给定向量和含重复元素的 pytest 测试均通过，未调用 `sorted`。

实例 9 用 AddressSanitizer 识别 use-after-free (Debugging)

题目要点：先正常编译并运行释放后继续访问内存的程序，再使用 AddressSanitizer 编译，阅读报告并修复问题。

```
char *greeting = malloc(32);
strcpy(greeting, "Hello, world!");
printf("%s\n", greeting);
free(greeting);
greeting[0] = 'J';           /* invalid: use after free */
```

```
$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
ERROR: AddressSanitizer: heap-use-after-free on address ...
#0 main uaf.c:10

# Fix: move free(greeting) to after the final use (or remove that use).
Hello, world!
Jello, world!
```

结果说明

普通运行可能表面正常，但 ASan 报出 heap-use-after-free 并定位到释放后的写入行。将 free 放到最后一次使用之后，内存生命周期与访问顺序一致，错误消失。

实例 10 比较两种自有实现的性能 (Profiling)

题目要点：比较同一任务的两种实现。本周第 8 题用同一份 words.txt 比较列表去重和集合去重；课程练习推荐 hyperfine，本环境沿用题目指定的 time 进行两次测量并取中位数，再用 cProfile 定位热点。

	run 1	run 2	median
list membership	1.765 s	1.749 s	1.7570 s
set membership	0.088 s	0.117 s	0.1025 s

count (both) = 1000
speedup = 1.7570 / 0.1025 = 17.14x

结果说明

cProfile -s cumulative 表明慢版本的主耗时在列表成员检查循环；将 unique 改为集合后，最终 count 不变为 1000，而中位耗时约降低为原来的 1/17，符合“先测量、再优化、再验证”的流程。

实例 11 定位并释放被占用的端口（Profiling）

题目要点：启动 `python -m http.server 4444`，在另一终端发现监听进程并终止它。课程 Linux 命令为 `ss -tlnp | grep 4444`；Windows 中以 `netstat -ano` 完成同样的端口-PID 映射。

```
PS> python -m http.server 4444
Serving HTTP on 0.0.0.0 port 4444 ...

PS> netstat -ano | findstr :4444
    TCP    0.0.0.0:4444    0.0.0.0:0      LISTENING      22432
PS> Stop-Process -Id 22432
PS> netstat -ano | findstr :4444
# no output
```

结果说明

监听端口先映射到 PID，再按 PID 终止进程；第二次查询无输出，证明 4444 已释放。这一流程可用于解决“端口已经被其他进程占用”导致的服务启动失败。

解题感悟

本周实验把“能运行”进一步推进到“可控制、可维护、可调试、可测量”。任务控制实验说明，结束后台进程时应优先发送可处理的信号并保留清理机会；语义重构和静态检查减少了手工同步修改的风险；调试器帮助直接观察索引状态，避免凭感觉改代码；性能优化则再次说明，应先用计时和分析工具找到瓶颈，再选择更合适的数据结构，并用相同输入验证结果与加速效果。

GitHub 仓库：

<https://github.com/FinaleQuill/system-dev-tools-foundation.git>